# Build Your First Telegram Bot

## A practical, step-by-step TeleBot tutorial

Start with an empty folder. Finish with a working bot that welcomes you, explains its commands, and repeats your messages.

**Alireza Khajehvandi**

Portfolio • GitHub • LinkedIn

YouTube • Telegram • Instagram

# How to use this tutorial

This is a hands-on guide. Complete the chapters in order and pause at every checkpoint. Commands are ready to copy, but read the short explanation beside each one so you know what it changes. You need no previous bot experience.

The English and Persian editions contain the same implementation, commands, expected results, and safety advice. Only their language and page direction differ.

**Important**

Never share a Telegram bot token. The examples use placeholders. Generated PDF is published on my github repo. but the LaTeX build files are intentionally excluded from Git because are not essential and can always be rebuilt from the source files.

# Contents

Chapter 1

# Meet the Bot We Will Build

**Chapter goal**

Understand the finished result, the few tools involved, and the project layout.

Our goal is intentionally small: a Telegram bot that is easy to finish and easy to understand. It will:

- welcome a user who sends `/start`;
- explain itself when the user sends `/help`;
- reply to ordinary text with `You said:` followed by that text;
- ask for text when it receives a photo, voice note, or similar content.

For example, when Alireza sends `My first bot`, the conversation is:

**Expected result**

**Alireza:** My first bot
**Bot:** You said: My first bot

## 1.1 The four pieces

Telegram shows the conversation. BotFather creates the bot account and gives its secret token. Python runs our instructions. The `pyTelegramBotAPI` package provides the `telebot` module that connects those instructions to Telegram. This tutorial uses polling: while the program runs, it regularly asks Telegram whether a new message has arrived.

## 1.2 A small, readable layout

```
telbot-simple-bot/
|-- code/
| |-- bot.py
| |-- requirements.txt
| |-- test_bot.py
| `-- README.md
```

```
|-- documentation/
| |-- English/
| `-- Persian/
|-- LINKEDIN_POST.md
`-- README.md
```

Only `code/bot.py` is needed to run the bot after the dependency is installed. The other files teach, test, or document it.

**Checkpoint**

You know exactly what the finished bot will do. No token, package, or account has been changed yet.

Chapter 2

# Prepare Python and the Project

**Chapter goal**

Create an isolated Python environment and install the one required package.

Install Python 3.10 or newer. Check it in Terminal, PowerShell, or the terminal inside your editor:

```
python --version
```

On some macOS or Linux systems the command is `python3`. A result such as `Python 3.12.4` is suitable.

## 2.1 Open the code folder

Clone the repository and enter its code folder:

```
git clone https://github.com/alirezaaies/telbot-simple-bot.git
cd telbot-simple-bot/code
```

If you downloaded a ZIP file, extract it and open a terminal in its `code` folder instead.

## 2.2 Create an isolated environment

An environment keeps this tutorial's package separate from other Python projects.

**macOS or Linux:**

```
python3 -m venv .venv
source .venv/bin/activate
```

**Windows PowerShell:**

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

The terminal usually adds `(.venv)` before its prompt. Then install the dependency:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The requirements file contains only `pyTelegramBotAPI`. Its import name is `telebot`; this difference is normal.

**Checkpoint**

Run the command below. If it prints `telebot is ready`, installation is complete.

```
python -c "import telebot; print('telebot is ready')"
```

Chapter 3

# Create the Bot with BotFather

**Chapter goal**

Create a Telegram bot account and make its token available safely to Python.

## 3.1 Ask BotFather for a bot

1. Open Telegram and search for the verified `@BotFather` account.
2. Start the chat and send `/newbot`.
3. Enter a display name, for example `Alireza Tutorial Bot`.
4. Enter a unique username ending in `bot`, for example `alireza_tutorial_123_bot`. If it is taken, choose another.
5. BotFather returns a token. Treat it like a password.

**Important**

Do not put the real token in source code, screenshots, chat messages, or Git. If it is exposed, send `/revoke` to BotFather and generate a new token.

## 3.2 Set the token for one terminal session

Replace the placeholder with the token BotFather gave you.

**macOS or Linux:**

```
export BOT_TOKEN="paste_your_token_here"
```

**Windows PowerShell:**

```
$env:BOT_TOKEN="paste_your_token_here"
```

The variable disappears when that terminal closes, which is safe and simple for a first project. You must set it again in each new terminal session.

**Checkpoint**

Check only whether the variable exists; do not print its secret value.

```
python -c "import os; print('token set' if os.getenv('BOT_TOKEN') else
    'missing')"
```

The expected result is `token set`.

Chapter 4

# Build and Understand the Bot

**Chapter goal**

Connect the token to TeleBot and understand how each message gets its reply.

The repository already contains the completed file below. Reading it from top to bottom is the fastest way to connect each idea to working code.

Listing 4.1: The complete code in code/bot.py

```
"""Run a small Telegram bot with the pyTelegramBotAPI (telebot) library.

The bot answers /start and /help, repeats normal text messages, and explains
what to do when it receives another content type. The Telegram token is read
from the BOT_TOKEN environment variable so that no secret is stored in code.

Author: Alireza Khajehvandi
Project: https://github.com/alirezaaies/telbot-simple-bot
"""

from __future__ import annotations

import os

import telebot
from telebot import types


def start_reply(first_name: str | None) -> str:
    """Return the welcome message used by the /start command."""
    name = first_name or "there"
    return (
        f"Hello, {name}!\n\n"
        "I am your first Telegram bot.\n"
        "Send me any text and I will repeat it.\n\n"
        "Commands:\n"
        "/start - show the welcome message\n"
        "/help - show usage help"
    )


```

```python
def help_reply() -> str:
    """Return a short, beginner-friendly help message."""
    return (
        "How to use this bot:\n"
        "1. Send any text message.\n"
        "2. The bot replies with the same text.\n"
        "3. Send /start whenever you want to see the welcome message again."
    )


def echo_reply(text: str) -> str:
    """Build the reply for a normal text message."""
    return f"You said: {text}"


def create_bot(token: str) -> telebot.TeleBot:
    """Create the bot and register all message handlers."""
    if not token.strip():
        raise ValueError("The Telegram bot token cannot be empty.")

    bot = telebot.TeleBot(token)

    @bot.message_handler(commands=["start"])
    def handle_start(message: types.Message) -> None:
        """Welcome the user and list the available commands."""
        bot.reply_to(message, start_reply(message.from_user.first_name))

    @bot.message_handler(commands=["help"])
    def handle_help(message: types.Message) -> None:
        """Explain how the bot behaves."""
        bot.reply_to(message, help_reply())

    @bot.message_handler(content_types=["text"])
    def handle_text(message: types.Message) -> None:
        """Repeat every ordinary text message."""
        bot.reply_to(message, echo_reply(message.text or ""))

    @bot.message_handler(
        content_types=["audio", "document", "photo", "sticker", "video", "
            voice"]
    )
    def handle_unsupported_content(message: types.Message) -> None:
        """Guide the user when this tutorial bot receives non-text content.""
            "
        bot.reply_to(message, "Please send a text message so I can repeat it.
            ")

    return bot


def main() -> None:
    """Read configuration and keep the bot connected to Telegram."""
    token = os.getenv("BOT_TOKEN")
    if not token:
        raise SystemExit(
            "BOT_TOKEN is missing. Export it in your terminal, then run bot.py
                again."
        )
```

```
86
87      bot = create_bot(token)
88      print("Bot is running. Press Ctrl+C to stop it.")
89      bot.infinity_polling(skip_pending=True)
90
91
92  if __name__ == "__main__":
93      main()
```

## 4.1 Step 1: import the tools

`os` reads the environment variable. `telebot` creates the client that talks to Telegram. `types` supplies message type hints; these hints help readers and editors but do not change the bot's behavior.

## 4.2 Step 2: keep replies easy to test

The functions `start_reply`, `help_reply`, and `echo_reply` only build text. They do not need the network. Separating them makes the expected conversation clear and lets the offline tests check it.

## 4.3 Step 3: register handlers

A *handler* is simply a function chosen for a kind of incoming message. The decorators above the functions define the choice:

- `commands=["start"]` handles `/start`;
- `commands=["help"]` handles `/help`;
- `content_types=["text"]` handles other text;
- the final handler covers common non-text messages.

Order matters: command handlers appear before the general text handler, so a command receives its special answer.

## 4.4 Step 4: start polling

`main` stops with a readable error when `BOT_TOKEN` is absent. When it exists, `infinity_polling` keeps listening until you press `Ctrl+C`. The `skip_pending=True` option ignores old messages that accumulated while this learning bot was offline.

**Checkpoint**

You can point to the exact handler used for `/start`, `/help`, a normal text message, and a photo.

Chapter 5

# Run It and Verify the Result

**Chapter goal**

Start the bot, complete a real Telegram conversation, and run offline checks.

Confirm that the environment is active and `BOT_TOKEN` is set, then run:

```
python bot.py
```

**Expected result**

```
Bot is running. Press Ctrl+C to stop it.
```

Leave that terminal open. In Telegram, open the link BotFather gave you or search for the username you created. Press **Start**.

## 5.1 A four-part manual check

1. Send `/start`. The reply begins with your Telegram first name and shows both commands.
2. Send `/help`. The reply gives three short usage steps.
3. Send `My first bot`. The reply must be `You said: My first bot`.
4. Send a photo. The reply must be `Please send a text message so I can repeat it.`

If all four results match, the complete path—Telegram, your token, Python, TeleBot, and the handlers—works.

## 5.2 Run the offline tests

Stop the bot with `Ctrl+C`, then run:

```
python -m unittest -v
```

**Expected result**

Four tests are listed, followed by `OK`. These checks use no token and do not contact Telegram; they verify the reply-building functions.

Whenever you change a message in `bot.py`, update its matching test and run this command again.

**Checkpoint**

Your bot answers all four manual checks, and the four offline tests pass.

Chapter 6

# Fix Common Problems and Continue

**Chapter goal**

Solve the most likely first-run problems and know what to learn next.

## 6.1 Quick problem guide

| What you see | What to do |
|---|---|
| `No module named telebot` | Activate the intended environment and run `python -m pip install -r requirements.txt`. |
| `BOT_TOKEN is missing` | Set the environment variable in the same terminal where you run `python bot.py`. |
| `Unauthorized` or token error | Copy the full token again without spaces. If it may have leaked, revoke it in BotFather first. |
| Bot prints that it runs but does not reply | Press Start in Telegram, confirm the username, keep the terminal open, and check the internet connection. |
| Another polling instance error | Stop the other running copy of this bot. Only one polling process should use the same token. |

## 6.2 Compile this book in TeXstudio

Open the edition's `main.tex`, choose **XeLaTeX** as the compiler, and build twice so the table of contents is current. Do not compile a chapter file by itself. The Persian edition requires XePersian and uses IRANSansX when installed, with B Nazanin and Noto Naskh Arabic fallbacks configured.

Generated files such as PDF, AUX, LOG, TOC, and SyncTeX output are ignored by Git. Keep them locally or attach a PDF to a release when you want to distribute a built copy; do not commit repeated build products to the repository.

## 6.3 Small next steps

After this version works, make one change at a time: add a `/about` command, customize the welcome text, add reply buttons, or deploy the bot on a service that can keep Python running. Never hard-code the token during deployment.

**Expected result**

You have built, understood, and checked a real Telegram bot. The project is small by design; it is now a safe foundation for your next feature.

# About the author

This tutorial and project idea were created by **Alireza Khajehvandi** to help new learners get a useful result quickly and understand every file they run.

Continue learning and get in touch:

- LinkedIn
- YouTube channel
- Telegram channel
- Instagram
- GitHub
- Personal website and portfolio